

Unit-3

Introduction to project management: The need for source code control, The history of source code management, Roles and code, source code management system and migrations, Shared authentication, Hosted Git servers, Different Git server implementations, Docker intermission, Gerrit, The pull request model, GitLab.

DevOps project management is a union between the two disciplines where project management is tailored to and supports the DevOps model. In a DevOps project management approach, project managers serve as coordinators between multiple contributors and as trackers of timelines and dependencies.

However, project managers also need to be closely aligned with the DevOps team and bring a deep understanding of the development process and skills needed to create the end product. There are several best practices that project managers should follow to achieve this integration with the DevOps pipeline.

The need for source code control

Source code control, also known as version control, is essential in software development. It allows teams to track and manage changes to code over time. Version control systems (VCS) enable developers to maintain a history of code changes. They provide a safety net by allowing teams to revert to previous code versions if needed. This feature is vital for troubleshooting and maintaining software quality.

Needs of VCS are:-

1. Collaboration and Teamwork

In DevOps, multiple developers often work on the same codebase simultaneously. Source code control systems like Git allow for seamless collaboration by enabling developers to work on different parts of the codebase concurrently without overwriting each other's changes. Branching and merging features allow teams to develop new features, fix bugs, or experiment with new ideas without disrupting the main codebase.

2. Change Tracking and Accountability

Source code control keeps a detailed history of all changes made to the code, including who made the changes, when they were made, and why.

This makes it easier to track progress, roll back changes if something goes wrong, and ensure accountability for every change made to the system.

3. Continuous Integration/Continuous Deployment (CI/CD)

DevOps emphasizes continuous integration (CI) and continuous deployment (CD), which require frequent code changes to be automatically tested and deployed. Source code control ensures that changes are integrated in a systematic manner, triggering automated build and test processes, which is essential for maintaining code quality and stability.

4. Versioning and Rollbacks

Version control allows teams to keep track of different versions of the software and manage multiple releases simultaneously. If a problem arises in production, it's easy to revert to a previous stable version of the code, minimizing downtime and reducing risk.

5. Auditing and Compliance

Many organizations need to comply with industry standards and regulations that require thorough auditing of code changes. Source code control systems provide detailed logs and change histories, ensuring that organizations can meet compliance requirements by demonstrating a well-documented development process.

6. Disaster Recovery

With version control, the source code is typically stored in distributed repositories, ensuring that in the event of hardware failures or other disasters, the code can be recovered from remote locations. This redundancy is crucial for minimizing downtime and ensuring the continuity of development.

7. Automation and Infrastructure as Code (IaC)

DevOps integrates infrastructure management with development processes. Source code control is not limited to application code but also includes Infrastructure as Code (IaC) files that define the infrastructure. This ensures consistent, repeatable deployments and allows teams to track infrastructure changes just like application code.

8. Security

- In a DevOps environment, where speed and agility are important, security can sometimes be overlooked. Source code

control helps enforce security best practices by allowing teams to review code changes through pull requests and code reviews. Additionally, sensitive information such as passwords or API keys can be flagged and removed from the history if accidentally committed.

Source code management

Source code management is the practice of tracking modifications to source code. Keeping a running history of the changes made to a codebase helps programmers, developers and testers ensure that they're always working with accurate and up-to-date code and helps resolve conflicts when merging code from multiple sources.

Source code management is important because, in work environments, multiple developers work in a shared codebase at the same time. With each developer working on a single feature, one dev could unknowingly make conflicting code changes — or one developer's saved edits could overwrite the other dev's changes.

Before source code management became widespread, few safeguards existed to prevent these scenarios. Each time a developer started work on a file, they could let teammates know to ensure no one else was working on it simultaneously. This was hardly foolproof. And when it worked, this method left no record of all the changes made over the course of a project — if a change resulted in a bug, there was no way to trace where it originated or determine how to fix it.

Source code management solves these problems by:

- Tracking each developer's changes.
- Highlighting conflicts.
- Preventing code overwriting.

Developers are alerted to any conflicts so they can address them before they are merged into the source code and potentially compromise the application. In the sections that follow, we'll take a closer look at how source code management works, what benefits it brings to software development, and which best practices to follow to get the most out of the process.

How source code management works

The heart of the source code management process is the code repository, a centralized server that stores all of an organization's code as well as the source code management system that tracks code changes for various development projects.

Copies of source code are stored as individual files. Developers "check out" a file when they want to make code revisions and receive a copy of that file in their local environment. At the same time, the original file in the repository is locked so that no one else can check out that piece of code and inadvertently overwrite the current developer's changes.

After the developer makes and tests their changes, they "commit" them, saving the revised code to the repository. This replaces the original file, increments the code version number — a process called "version control" — and removes the lock.

The process is repeated every time another developer makes code changes to any file. The source code management system then stores and manages all revisions to maintain a complete history of each piece of code.

Regardless of tools, source code management uses an array of common terms, including the following:

- **Repository:** The server where a project's collected files are stored, as well as the source code management system used to track code changes. Developers "check out" code files to make changes, then save them back to the repository when their development work on that code is completed. The source code management system saves a new version of the file without replacing the old one.
- **Check out:** The act of taking a file from the repository to work on, which automatically locks the file so no other developer can update it.
- **Check in:** Refers to saving the checked-out file back to the repository once changes have been completed, which removes the lock.
- **Commit:** Another word for "check in."
- **Trunk:** The main development line of a file tree under version control.
- **Branch:** A copy of code created off a project's trunk. Branching allows multiple copies of the same file to be worked on in different ways, independently of each other.
- **Merge:** The integration of two sets of changes into a single file, such as joining a branch back to the trunk or joining two branches together. There

are numerous merge strategies to consider; one such strategy is rebase, which allows you to rewrite commits from one branch into another.

Source code management vs. version control

Version control is a revision control system designed to track changes to code over time. The term is often used interchangeably with source code management.

Version control is managed with version control software that tracks every code change in a special type of database. When developers save changes made to a file, the system retains — rather than overwrites — all the changes from every previous version of that file.

Maintaining a versioned record of all code changes in this way offers several benefits, as we'll see below. As part of the branch strategy, there are two types of version control systems: centralized and distributed.

- Centralized version control systems maintain one copy of a project on a centralized server to which all code changes are committed. Developers can check out the specific files they need to work on, but never have a full copy of the project locally.
- Distributed version control systems allow developers to clone the repository to their own local server or machine so they have the complete history of the project. Devs then clone the code they need to work on from this local copy of the master repository. When they're done with their changes, they commit them to their local source code repository, then “push” this local repository code to the master repository.

Each type of version control system has advantages and disadvantages.

Centralized version control is simpler to set up, less complex and easier to learn than distributed version control. It's also easier to control access because code is stored on a single server.

Performance is generally faster in distributed version control systems because you don't have to be connected to the network, which potentially slows connections. You also don't have to wait for locks to clear to access the code because the complete codebase is already on your local system. And if the main server goes down in a distributed version control system, you still have the complete code history saved and stored locally to serve as a backup.

Best Practices for Source Code Management

To get the most out of source code management, it's important to follow a few best practices:

- **Commit often.** Every commit is a snapshot of the codebase at a particular point in time. Frequent commits allow you to incrementally save completed changes to the codebase, which gives you more opportunities to revert to an earlier state if a change results in an error.
- **Include a commit message.** A commit message communicates the context of a change to other developers, and is often helpful for troubleshooting should problems arise from particular changes. Development teams have different conventions for writing commit messages, but generally they include a short line of text specifying the type of commit (e.g., new feature, bug fix, refactoring) and more detailed body text explaining what changes were made and why. For pipeline analytics, the commits should include backlog ticket numbers.
- **Work on the current version.** With multiple developers making rapid code updates, your local copy of the codebase can easily fall behind the master copy. Check out the latest codebase before making changes so you're working with the most up-to-date version. Moreover, versioning for microservices and CI/CD pipelines can be very different.
- **Standardize team workflows.** Every developer has their own preferred processes on projects. But for SCM tools and procedures to be effective, development teams need to establish consistent workflows which creates the foundation for a better product.
- **Use branches.** Branches create separate lines of development to allow developers to work in parallel on the same codebase without impacting each other's work. They should be used frequently and adhere to defined branch practices and strategies (creating a new branch for every change, for example) to ease integration into the main development line.

Source code management is essential for modern software development

As projects and the size of their respective codebases grow in number and complexity, source code management is becoming a crucial tool for software development teams. Modifying the wrong version of an app — or worse, releasing it with bugs you were supposed to fix — carries heavy costs in the form of lost time, money and customer loyalty.

Source code management is easy to implement, and when done effectively, can accelerate your development process, increase team productivity and improve the quality of your product.

Different Job Roles in DevOps :

1. DevOps Engineer :

In the prime of the organic phenomenon authority all told the items DevOps. Additionally, referred to as a platform Engineer, the DevOps engineer is that the one who oversees and supports the platform used for DevOps operation .

A qualified candidate for a DevOps engineer position has older in DevOps work, familiarity with the platform on the market and the way to integrate them into existing business processes, and management expertise

2. Build Engineer :

The build engineer may be a DevOps social control position to blame of the event groups. This individual spends their time managing the build and development method and is accountable for the guaranteeing build goals and deadlines square measure met in an exceedingly quick paced atmosphere. They managed code, maintain builds, they produce new build , and additionally manages deploy automatic answer, ensures build meet established configuration needs.

They have a hybrid role that features the responsibilities of different technical roles like the wife designer or the configuration manager. All these works for a large vary of prospects within the business having a great deal of flexibility within the position.

3. DevOps Evangelist :

In the market there square measure already several firms dedicated to DevOps lifestyle. These firms square measure guided by DevOps evangelist. The position holder or adviser or associate worker. This role desires authority as a result of their goal is develop an idea, a thought, a concept or an inspiration} for DevOps implementation and persuade those to blame of what proportion higher life are if that plan is enforced. A DevOps Evangelist is associate analyst and should be sensible at compiling and presenting information.

4. Release Manager :

If the build Engineer is that the back of the coin, the discharge manager is within the front. The discharge manager is associate oversight and social control

position that oversees the general development, they work consequently build engineers and different groups to make sure that the goals square measure met.

The release manager ought to be initial and foremost, a management and oversight position that needs a deep level of sensible development information with success.

5. Product manager :

DevOps is essentially targeted on conveyance operation system into development and most DevOps roles square measure development roles with enlarged responsibilities and skillsets. Its associate operation role repurposed among DevOps. This role is one in all that comes straight from operation aspect of DevOps and brings with it the structure and management skills necessary for the comes to succeed.

6. DevOps code developer :

The role of the code developer is crucial for any of code targeted company. In DevOps a code developer isn't simply accountable for developing code, however additionally for check, deploy, monitor, and support that code. The DevOps code developers ought to be fluent in cryptography and code development in addition as QA and maintenance.

7. Security Engineer :

Fashionable code development is fraught thus in DevOps, security is resistance and vital facet of the event method. Security has to be baked into the merchandise from the bottom up thus its harder to take advantage of and more durable to crack. This needs a security engineer to figure closely with the code developers, that is extraordinarily vital not only for the success of a product however additionally for the safety of the user exploitation it. It has to fluent with all fashionable security standards, in progress attacks ,vectors, and users behavior that generally bypass security and make holes.

SCM System and Migrations

A Source Code Management (SCM) system is a tool used to track changes made to code, collaborate with others, and manage different versions of code. Examples of SCM systems include:

1. Git
2. Subversion (SVN)
3. Mercurial

4. Perforce

Migrations, in the context of SCM, refer to the process of moving code from one SCM system to another, or upgrading to a newer version of the same system.

Types of migrations:

1. SCM system migration: Moving code from one SCM system to another (e.g., from SVN to Git).
2. Version upgrade: Upgrading to a newer version of the same SCM system (e.g., from Git 2.x to Git 3.x).

Best practices for migrations:

1. Backup: Backup your code and metadata before migration.
2. Test: Test the migration process in a non-production environment.
3. Plan: Plan for downtime and notify stakeholders.
4. Automate: Automate the migration process where possible.
5. Verify: Verify the integrity of the migrated code and metadata.

Tools for migrations:

1. Git-SVN: For migrating from SVN to Git.
2. svn2git: For migrating from SVN to Git.
3. git-fast-import: For migrating from other SCM systems to Git.
4. Perforce Git Fusion: For migrating from Perforce to Git.

Benefits of migrations:

1. Improved collaboration: Newer SCM systems often have better collaboration features.
2. Enhanced security: Newer SCM systems may have better security features.
3. Increased flexibility: Newer SCM systems may offer more flexibility in workflow and customization.
4. Better performance: Newer SCM systems may offer improved performance and scalability.

Shared authentication

Shared authentication in DevOps refers to the practice of using a single authentication mechanism across multiple tools and systems in the DevOps pipeline. This approach aims to simplify authentication, reduce complexity, and improve security.

Benefits of Shared Authentication in DevOps

1. **Simplified Access Management:** Shared authentication streamlines access management by providing a single point of authentication for multiple tools and systems.
2. **Improved Security:** Consistent authentication and authorization across tools reduce the risk of unauthorized access and improve overall security.
3. **Enhanced Collaboration:** Simplified access to shared resources and tools enhances collaboration among teams and improves productivity.
4. **Reduced Administrative Burden:** Centralized user credential management reduces the administrative burden of managing multiple credentials.
5. **Better Compliance:** Shared authentication helps organizations meet compliance requirements by providing a clear audit trail of user access and activity.

Methods of Shared Authentication in DevOps

1. **OAuth:** An industry-standard authorization framework that allows users to grant third-party applications limited access to their resources.
2. **OpenID Connect (OIDC):** Builds on top of OAuth to add authentication capabilities, providing a simple and secure way to authenticate users.
3. **LDAP (Lightweight Directory Access Protocol):** A centralized directory service for authentication, providing a single source of truth for user identities.
4. **Kerberos:** A secure authentication protocol for distributed systems, providing strong authentication and encryption.
5. **SAML (Security Assertion Markup Language):** An XML-based standard for exchanging authentication data, enabling single sign-on and identity federation.

Tools that Support Shared Authentication in DevOps

1. Jenkins: Supports various authentication methods, including OAuth and LDAP.
2. GitLab: Offers built-in OAuth and LDAP support for authentication and authorization.
3. GitHub: Supports OAuth, OIDC, and SAML for authentication and authorization.
4. Docker: Supports OAuth and LDAP for container registry authentication.
5. Kubernetes: Supports OIDC and LDAP for cluster authentication and authorization.
6. CircleCI: Supports OAuth and GitHub authentication for continuous integration and delivery.
7. Travis CI: Supports OAuth and GitHub authentication for continuous integration and delivery.

Best Practices for Implementing Shared Authentication in DevOps

1. Choose the Right Authentication Method: Select an authentication method that meets your organization's security and compliance requirements.
2. Implement Single Sign-On (SSO): Provide users with a seamless SSO experience across multiple tools and systems.
3. Use Multi-Factor Authentication (MFA): Add an extra layer of security by requiring users to provide additional verification factors.
4. Monitor and Audit User Activity: Regularly monitor and audit user activity to detect and respond to security incidents.
5. Continuously Review and Update Authentication Policies: Regularly review and update authentication policies to ensure they remain effective and compliant with changing regulations.

Hosted Git Server

A hosted Git server is a cloud-based or on-premise service that stores Git repositories, manages access control, and provides tools for collaboration, issue tracking, and CI/CD integration. These servers allow teams to work together on code, share changes, and version control across distributed systems.

Popular Hosted Git Servers:- GitHub, GitLab, Bitbucket, AWS Code Commit, Azure Repos.

.GitHub: GitHub is the most popular hosted Git service, widely used for open-source and private projects.

GitLab: GitLab offers a comprehensive DevOps platform with Git hosting, CI/CD pipelines, and integrated DevOps tools.

Bitbucket: Bitbucket, part of the Atlassian ecosystem, is another popular hosted Git service that integrates with Jira for issue tracking and project management.

AWS CodeCommit: AWS CodeCommit is a fully managed Git server hosted in AWS.

Azure Repos: Azure Repos is a Git hosting service provided by Microsoft's Azure DevOps platform.

Key Features of Hosted Git Servers in DevOps:

1. Version Control:

- Centralized code storage with Git's distributed version control allows teams to work on features simultaneously.
- Branching and merging facilitate parallel development.

2. Collaboration Tools:

- Hosted Git servers provide built-in tools like pull requests, code reviews, and inline comments to enhance team collaboration.
- Issue tracking and project management tools enable efficient team workflows.

3. CI/CD Integration:

- Hosted Git servers typically integrate with CI/CD tools (e.g., Jenkins, CircleCI, GitHub Actions, GitLab CI) to automate the building, testing, and deployment of applications.
- This automation ensures faster delivery and higher code quality.

4. Security and Access Control:

- Access control and role-based permissions ensure only authorized users can view, edit, or deploy code.
- Built-in security features like vulnerability scanning, secret management, and dependency analysis are often available.

5. Scalability and Performance:

- Hosted Git servers offer scalability, ensuring that large teams and enterprises can manage repositories and workflows without performance degradation.
- Features like caching, load balancing, and distributed repositories improve performance and reliability.

6. Backup and Disaster Recovery:

- Hosted services usually provide automated backups and disaster recovery solutions, ensuring that repositories are safe and retrievable in case of data loss.

4. Choosing the Right Git Hosting for DevOps:

When choosing a hosted Git server for your DevOps environment, consider the following factors:

- Integration with existing tools: Ensure the Git server integrates with your current CI/CD tools, cloud providers, and project management tools.
- Scalability: Consider the size of your team and whether the platform can scale to meet growing needs.
- Security: Check if the platform offers enterprise-grade security features like role-based access, audit logs, and security scanning.
- Cost: Evaluate the cost of the hosted Git server, especially if you need additional features like CI/CD or enhanced security.
- Support for self-hosting: Some organizations may prefer self-hosting for security or compliance reasons, so consider platforms like GitLab, Bitbucket Server, or AWS CodeCommit.

5. Self-Hosted Git Servers:

For organizations that require more control over their infrastructure or have strict compliance needs, self-hosting a Git server might be the best option.

Popular solutions include:

- GitLab (Self-hosted): Full control over the environment, suitable for larger organizations.
- Gitea: Lightweight, open-source Git hosting platform for self-hosting.
- Gogs: A simple, self-hosted Git service similar to GitHub for small teams.

Git server implementations

A Git server is a centralized platform that hosts and manages Git repositories, allowing multiple developers to collaborate on code development. It provides a single source of truth for code changes, enabling version control, collaboration, and tracking of changes.

Key features of a Git server:

1. Repository management: Hosts and manages multiple Git repositories.
2. Version control: Tracks changes to code, allowing for branching, merging, and reverting.
3. Collaboration: Enables multiple developers to work on the same repository simultaneously.
4. Access control: Manages user permissions, ensuring only authorized users can access and modify repositories.
5. Backup and recovery: Provides regular backups and recovery options to prevent data loss.
6. Security: Offers features like encryption, firewalls, and authentication to protect code and data.
7. Scalability: Supports large numbers of users, repositories, and projects.
8. Integration: Integrates with other development tools, such as continuous integration/continuous deployment (CI/CD) pipelines, issue trackers, and project management software.

Popular Git server options include:

1. GitLab
2. GitHub Enterprise
3. Bitbucket Server
4. Azure DevOps
5. AWS CodeCommit
6. Google Cloud Source Repositories
7. Phabricator
8. Gitea
9. GitKraken
10. Bonobo Git Server

Git server implementations used in DevOps, each with their own strengths and weaknesses. Here are some of the most popular ones:

1. GitLab: A popular, open-source Git server that offers a wide range of features, including issue tracking, continuous integration, and continuous deployment.
2. GitHub Enterprise: A self-hosted version of the popular GitHub platform, offering advanced security and compliance features.
3. Bitbucket Server: A commercial Git server developed by Atlassian, offering features like pull requests, code reviews, and continuous integration.
4. Azure DevOps: A cloud-based platform that includes a Git server, along with features like continuous integration, continuous deployment, and agile project planning.

5. AWS CodeCommit: A cloud-based Git server offered by Amazon Web Services, integrating with other AWS services like CodeBuild and CodePipeline.
6. Google Cloud Source Repositories: A cloud-based Git server offered by Google Cloud, integrating with other Google Cloud services like Cloud Build and Cloud Deploy.
7. Phabricator: An open-source software development platform that includes a Git server, along with features like code review, continuous integration, and project management.
8. Gitea: A lightweight, open-source Git server that offers features like issue tracking, pull requests, and continuous integration.
9. GitKraken: A commercial Git server that offers features like pull requests, code reviews, and continuous integration, along with a graphical user interface.
10. Bonobo Git Server: An open-source Git server that offers features like pull requests, code reviews, and continuous integration, with a focus on simplicity and ease of use.

When choosing a Git server implementation, consider factors like scalability, security, ease of use, and integration with other DevOps tools.

Docker intermission

Docker is a tool that's positioned between developers and operations personnel. Thanks to Docker, developers can hand over to the operations team an application - packaged as an image - that can run seamlessly on any environment (testing, staging, or production). It guarantees that if a feature functions in the development environment, it will also work in the production and staging environments.

Docker eliminates friction between the two teams and eases the work of automating steps such as testing, staging, and deployment. This helps accelerate

application development and improves the overall performance of applications in a production environment.

The Benefits of Using Docker:

Below are some of the benefits we get from using Docker:

- High ROI and cost savings
- Productivity and standardization
- Maintenance and compatibility
- Rapid deployment
- Faster configurations
- Seamless portability
- Continuous testing and deployment
- Isolation, segregation, and security

In DevOps, "**Docker intermission**" might refer to a break or pause in the workflow that involves Docker. Docker is widely used in DevOps for creating, managing, and deploying containerized applications. An intermission or pause could happen for various reasons, such as maintenance, upgrades, or troubleshooting in the Docker environment.

Here are a few contexts where a Docker intermission might occur:

1. CI/CD Pipeline Maintenance:

- If a CI/CD pipeline relies on Docker containers to build, test, and deploy applications, there might be an "intermission" when the Docker environment is being updated or maintained.

2. Container Upgrades:

- During the upgrade of Docker containers or Docker engine, there might be a short intermission to ensure no service disruptions. For example, upgrading base images or Docker versions could cause a planned pause in the containerized workflow.

3. Scaling Down or Migration:

- In a DevOps setting, scaling down services or migrating Docker containers between clusters could require an intermission to avoid service interruptions.

4. Resource Constraints:

- Sometimes, during high resource utilization, developers might pause Docker-based workflows to allocate resources properly before continuing with container operations.

5. Troubleshooting and Debugging:

- In case of issues like container crashes, networking issues, or dependency conflicts, an intermission in Docker operations might be needed to debug and resolve the problems.

6. Automated Builds and Deployment Intermissions:

- In automated environments, a pause in Docker-related tasks might be necessary to sync with external services, API rate limits, or testing suites.

Gerrit

Gerrit is a web-based, open-source code review tool that integrates with Git. It allows teams to review changes in a collaborative manner before the code is merged into the main repository. Gerrit helps streamline the code review process, ensuring that code quality is maintained and that any potential issues are caught early in the development cycle.

History of Gerrit

Gerrit was originally developed by Google for its own internal projects.

It was released as an open-source project in 2010.

Since then, it has gained a significant user base in the open-source community.

Key Features of Gerrit:

1. **Code Review Workflow:** Gerrit facilitates a structured code review process where changes can be submitted for review, discussed, and approved or rejected based on feedback from reviewers.
2. **Integration with Git:** Gerrit works as a Git repository, allowing developers to push changes to Gerrit instead of directly to the main

branch. The changes go through a review process where they can be refined and improved.

3. **Access Control:** Gerrit provides fine-grained access control, allowing teams to define who can view, review, and submit changes to specific projects or branches.
4. **Change Management:** Each change in Gerrit is tracked with its own review page, where discussions, comments, and reviews can be made. Changes can be amended, and new versions can be submitted in response to feedback.
5. **Automated Checks:** Gerrit can integrate with continuous integration (CI) systems to automate testing, linting, and other checks. This ensures that changes meet the project's standards before they are merged.
6. **Branch and Project Management:** Gerrit supports the management of multiple branches and projects, making it easier to handle complex workflows in larger codebases.

Architecture of Gerrit

Gerrit is built on a Java-based architecture that interacts with Git.

The system consists of a web application and a server-side component.

It uses a PostgreSQL database to store configurations and review data.

User Roles in Gerrit

1. Administrators

Administrators have full control over the Gerrit instance. They can manage projects, users, permissions, and settings.

Responsibilities:

- Create, configure, and delete projects.
- Manage user accounts and groups.
- Define and modify access controls and permissions.
- Configure Gerrit settings and integrations with other tools like CI/CD pipelines.
- Troubleshoot and resolve any issues with the Gerrit instance.

2. Project Owners

Project Owners have administrative rights over specific projects but not the entire Gerrit instance.

Responsibilities:

- Manage access controls for their specific projects.
- Add or remove users from project-specific groups.
- Configure project-specific settings and integrations.
- Approve changes that affect project settings or structure.

3. Developers (Contributors)

Developers are the primary users who contribute code to the projects.

Responsibilities:

- Submit changes (patch sets) for review.
- Respond to feedback by amending changes as needed.
- Review changes submitted by others, if granted review permissions.
- Work on assigned changes and ensure that their submissions meet project standards and pass any automated checks.

4. Reviewers

Reviewers are responsible for reviewing and approving or rejecting changes submitted by developers.

Responsibilities:

- Review code changes, comment on specific lines, and suggest improvements.
- Score changes using Gerrit's scoring system (e.g., +1, -1, +2, -2).
- Approve changes that meet the project's quality standards.
- Reject or request additional changes if the submission does not meet the required standards.

5. Integrators (Submitters)

Integrators are responsible for merging approved changes into the main branch.

Responsibilities:

- Ensure that changes meet all approval and verification criteria before submission.
- Submit changes once they have the required approval scores from reviewers and pass automated checks.
- Monitor the health of the main branch and address issues related to submitted changes.

6. CI Bots

Continuous Integration (CI) Bots are automated users that perform tasks like running tests, checks, and other automated processes.

Responsibilities:

- Automatically verify changes by running tests and checks.
- Provide feedback on whether changes pass or fail these checks.
- Prevent changes from being merged if they do not pass the required checks.

Advantages of Using Gerrit:

- **Improved Code Quality:** By ensuring that all changes go through a review process, Gerrit helps maintain high code quality.
- **Collaboration:** It fosters collaboration among team members, as feedback is shared and discussed directly in the review process.
- **Transparency and Traceability:** Every change, comment, and review is logged, providing a clear history of the decision-making process.
- **Customizable Workflow:** Teams can customize the review and approval process to fit their specific needs.

The Pull Request

A **pull request** – also referred to as a merge request – is an event that takes place in software development when a contributor/developer is ready to begin the process of merging new code changes with the main project repository.

A project's repository (or "repo") is where all the project code, product knowledge, metadata, and much more data/information is stored. A repository maintainer/manager is someone on a DevOps or operations team who is responsible for maintaining said repository. They are in charge of which updates can be merged with the final project and ultimately made available to end users.

During a pull request, the repository maintainer reviews new code updates from a developer to determine whether or not it is ready to be released. Without pull requests, unfinished or incorrectly written code updates could be prematurely merged with the main repository and break or cause issues with the live product. Pull requests ensure the integrity and user experience of the product by only merging code that has been appropriately reviewed and approved.

In addition to protecting the main project repository, pull requests also encourage collaboration and open communication when working on new product updates. They also help to keep teams motivated by highlighting and notifying the team when someone completes a new feature.

How do Pull Requests Work?

A pull request works by allowing developers to create new features or squash bugs without affecting the main project code or what the users are seeing. This way, they are able to write and test code changes locally without having to worry about breaking the overall product.

Pull requests follow a basic five step process:

- **Fork Main Repository and Create a Local Clone.** First, the developer creates a fork of the main repository, and then clones this onto their local machine.
- **Make Needed Changes Locally.** The developer then is able to make their needed changes or additions to the code whether they are working on resolving an issue or new feature.
- **Push Local Changes to Forked Repository.** Once the developer has completed and tested the new code changes, they push these changes back to the forked repository they created in step one.

- **Make a Pull Request.** This is where the actual pull request takes place! After requesting a pull request, the main repository maintainer is alerted for review. The maintainer will then review the work done in the developer's forked repository, and then make any comments or request any edits that need to be made for approval.
- Any edits are then sent back to the developer for additional commits (changes to code) that may be needed.
- If no edits are needed, the pull request is approved by the maintainer.
- **Merge with Main Project.** Once the repository maintainer has approved a pull request, the developer's new updates in the forked repository are merged with the main project repository. The product is then updated with the new feature or bug fix, and can now be viewed by end users.

In some cases, a developer may also make a pull request for a feature or update that is not yet completed. This way, if a developer is stuck on a new update they've been working on, they can now get feedback from other team members and work through possible solutions.

In either case, using pull requests ensures that any new update for a given project has been thoroughly reviewed and approved before it is merged with the main repository. This helps to prevent future issues and ensures a seamless user experience with minimal downtime.

Creating a Pull Request

Creating pull requests may vary from organization to organization based on the tools being used and the type of repository (ex: GitHub, BitBucket, etc.). However, creating a pull request often involves these same three elements:

Drafting

When creating a pull request, the developer will first begin with a draft. Here, they will be able to title and briefly describe their code changes. They will need to include information about the type of update (whether it's a new feature or a bug fix) as well as its source repository/branch and destination repository/branch.

A draft pull request cannot be merged until the developer has marked it as ready for review.

Merging

Merging occurs once a developer's submitted pull request has been approved by the repository maintainer. Before merging, the repository maintainer will need to review the developer's completed work. They will comment on any issues, and request edits from the developer as needed. Once any and all updates have been made, the repository maintainer can then safely merge the updates with the main project repository, making it live for end users.

Updating

If the repository maintainer requires any changes to the code updates before merging, the developer will be alerted and provided with any comments or feedback for fixing the issue. Once the developer has resolved the issue, they can update the pull request with their new commits added for further review and approval.

Pull Request Templates

Pull request templates can be created and stored to help expedite the process and ensure consistency across all team members. When creating your pull request template, it's important to include:

- **Summary of Code Update:** Was this a bug fix or a new feature? Summarize what was accomplished with this new code update.
- **Unit Test Cases:** Was this code update tested across all devices and browsers?

The specifics of your pull request template will vary based on the repository your organization uses. Different tools have different capabilities and options to make pull requests a breeze for your team.

An Example of a Pull Request

Now that we've gone through what a pull request is and how they work, let's take a look at a possible pull request in action.

1. Russell is a developer at a software company that has a mobile application that lets users rate and review different wines.
2. Russell needs to create a new feature that allows the user to highlight their "Five Favorite Wines"
3. To begin working on this new feature, Russell first creates a fork of the main repository and clones this to his local machine.

4. Russell then develops this new feature locally.
5. Once completed, Russell can push his code updates to the forked repository he created.
6. Now Russell can create a pull request!
7. Once the pull request has been submitted, Erica (the Repository Maintainer) is alerted to begin reviewing Russell's new code update.
8. Erica can then request any follow-up commits from Russell as needed.
9. When any and all change requests are made, Erica will then approve Russell's pull request.
10. Russell's new update is then merged with the main project repository, and application users can now begin using the new application feature.

GitLab

GitLab is a comprehensive, web-based DevOps platform that provides a complete suite of tools for managing the software development lifecycle, from planning and source code management to CI/CD, monitoring, and security. It is designed to streamline and enhance the collaboration and productivity of development teams by providing everything needed to build, deploy, and maintain software in one place.

Key Features of GitLab:

1. **Source Code Management (SCM):** GitLab provides robust Git repository management, including branching, merging, and pull requests, all integrated into a single platform. It supports features like code reviews, issue tracking, and merge requests, which facilitate collaboration and code quality.
2. **Continuous Integration and Continuous Deployment (CI/CD):** GitLab has built-in CI/CD pipelines that allow teams to automate the building, testing, and deployment of code. This enables developers to catch issues early and deliver features faster and more reliably.
3. **DevSecOps:** Security is integrated into GitLab's workflow, offering tools for static and dynamic application security testing (SAST/DAST), dependency scanning, container scanning, and more. This helps teams identify and address security vulnerabilities throughout the development process.

4. **Project Management:** GitLab includes a range of project management tools, such as boards, milestones, and issue tracking. These tools help teams plan, track progress, and manage tasks efficiently.
5. **Collaboration Tools:** GitLab facilitates collaboration through features like merge requests, inline commenting on code, discussions, and mentions. This ensures that team members can work together effectively, whether they are co-located or distributed.
6. **Code Review and Quality Assurance:** GitLab supports code review processes through merge requests, where reviewers can provide feedback, suggest changes, and approve or reject code. Automated testing and code quality checks can be integrated into the workflow to maintain high standards.
7. **Container Registry:** GitLab includes a built-in container registry that allows teams to store and manage Docker images, which can be used in CI/CD pipelines or for deployment.
8. **Monitoring and Logging:** GitLab provides tools for monitoring application performance and tracking logs. It integrates with Prometheus for monitoring and can be used to set up alerts and visualize application metrics.
9. **Integration with Other Tools:** GitLab integrates with numerous third-party tools and services, including cloud providers (AWS, Azure, GCP), Kubernetes, Jira, Slack, and many others, making it highly adaptable to various workflows.

GitLab Editions:

GitLab is available in multiple editions:

- **GitLab Community Edition (CE):** This is the open-source version of GitLab, which provides core features suitable for small to medium-sized teams and projects.
- **GitLab Enterprise Edition (EE):** This version includes advanced features tailored for larger enterprises, such as enhanced security, performance optimizations, and additional integrations. EE is available in different tiers, including Premium and Ultimate, each offering progressively more features.

Advantages of Using GitLab:

- **All-in-One Platform:** GitLab provides a single application for the entire DevOps lifecycle, reducing the need for multiple tools and simplifying integrations.
- **Scalability:** GitLab is suitable for small teams to large enterprises, with features and pricing tiers that scale according to the needs of the organization.
- **Automation and Efficiency:** GitLab's CI/CD pipelines and automation capabilities help teams move faster by reducing manual tasks and increasing deployment frequency.
- **Strong Security Posture:** With integrated security scanning and compliance features, GitLab helps teams build secure applications and maintain regulatory standards.
- **Open Source Community:** GitLab's Community Edition is open-source, allowing for community contributions, transparency, and flexibility in usage.

Use Cases:

- **Software Development:** GitLab is widely used by development teams to manage source code, track issues, and collaborate on code reviews.
- **DevOps Automation:** Teams use GitLab to automate the build, test, and deployment process, making it a cornerstone of DevOps practices.
- **Security and Compliance:** With integrated security tools, GitLab helps organizations maintain secure codebases and meet compliance requirements.